

CODA-SC: Calibrated Selective Edge-Cloud Inference Under Internet Deadlines

Anonymous Authors

Abstract

Edge-cloud inference must decide whether to answer locally, offload remotely, or use a local fallback under predictive uncertainty and variable Internet latency. Existing split-computing methods mainly optimize profiled systems costs, whereas early-exit methods often rely on uncalibrated confidence thresholds. We propose CODA-SC, a calibrated selective edge-cloud inference framework for multi-exit CNNs. CODA-SC accepts a local exit only when the conformal prediction set is a singleton; otherwise, a primal-dual controller selects a remote split or local fallback using latency, upload, energy, and deadline feedback. We establish finite-sample coverage and selective-error bounds for the exits and analyze an idealized binary-slack controller with $O(\sqrt{T})$ regret and constraint violation under bounded feedback. Using measured CNN FLOPs and activation payloads with simulated Internet-like traces across four vision workloads, four network profiles, five seeds, and twelve policies, CODA-SC-Cov reduces mean latency from 109 ms to 26.8 ms and deadline misses from 52% to 0.05%, with 0.015 KB upload and about a two-point accuracy reduction. CODA-SC-Risk trades latency for lower selective error and higher accuracy. A learned-bottleneck split study further tests compressed intermediate actions while retaining the device-model and trace-simulation scope. These results support calibrated selective control, not hardware deployment validation.

Keywords: split computing; conformal prediction; edge intelligence; online optimization; reliable machine learning; intelligent systems

Highlights

- Conformal exits calibrate selective edge-cloud inference.
- A primal-dual controller handles deadline-critical residual samples.
- CNN FLOPs and activation payloads are measured, not hand-set.
- Learned bottlenecks test compressed intermediate split actions.
- Reproducible code regenerates tables, figures, and diagnostics.

1 Introduction

Edge-cloud inference systems must decide where a neural-network request should be executed. Cloud-only inference preserves the accuracy of the full model, but each request incurs uplink transmission, round-trip delay, queueing, and loss. Local-only inference avoids network uncertainty, but it consumes device compute and energy. Split computing introduces intermediate actions: execute

a prefix on the device, transmit the activation, and complete the suffix remotely. Multi-exit networks add sample-adaptive local decisions, allowing easy examples to terminate before either the full local model or the network is used.

The standard formulation treats these decisions separately. Split-computing systems such as Neurosurgeon [9], Edgent [12], and Auto-Split [4] profile layer compute, payloads, and bandwidth to select a partition. Early-exit systems classify samples at intermediate heads when a confidence score exceeds a threshold [15, 19, 20]. Online partitioning methods further adapt to unknown delay [17]. These lines of work address important parts of the pipeline, but they leave a coupled reliability question unresolved: when an intermediate classifier is uncertain and the network is also uncertain, the system must decide whether to exit, offload, or pay for a local full-model fallback.

This coupling matters because the two uncertainties fail in different ways. A confidence threshold can have favorable empirical accuracy on calibration data without finite-sample coverage on future samples. A split policy can minimize expected latency while still violating a hard deadline during queueing spikes. The resulting deployment failure is not simply excessive mean latency; it is a request-level decision error in which the system either trusts an unreliable early prediction or sends an uncertain sample into a network path that is already unlikely to meet the deadline.

We study this decision as a selective-prediction and constrained online-control problem. CODA-SC wraps each intermediate exit in split conformal prediction [2], so a local prediction is accepted only through a calibrated prediction-set rule. Samples not accepted by the exits are assigned by a primal-dual controller over remote split points and a local full-model fallback. The controller estimates latency from measured local compute, activation payload, bandwidth, RTT, queueing, and cloud suffix compute, and uses a virtual deadline queue to penalize actions that are likely to exceed the deadline budget. Thus, statistical uncertainty determines whether an early exit is eligible, while online network feedback determines how to serve the remaining requests.

The empirical evaluation is designed to avoid a common ambiguity in split-computing studies: conclusions driven by hand-set payload and compute constants. We train multi-exit convolutional networks on four vision workloads. Activation payloads are exact int8 tensor sizes, and compute latency and energy are derived from measured per-stage FLOPs under a documented edge/cloud device model. Network variability is evaluated with simulated Internet-like traces that include bandwidth, RTT, queueing, and loss; therefore the absolute network latencies should be interpreted as trace-simulation results, while the model-side costs are measured from the trained networks.

The resulting contribution is positioned as an intelligent-systems decision method rather than a network protocol. CODA-SC combines uncertainty quantification, online optimization, and resource-aware deep inference: conformal prediction specifies when an intermediate representation is informative enough to leave the device as a label, while the controller determines when the information should be transmitted, retained locally, or completed by the full-model fallback. The contribution therefore lies at the interface between machine learning, information processing, and adaptive computational-resource allocation, rather than in network protocol design alone.

This paper makes the following contributions.

- We formulate deadline-constrained edge-cloud inference as a joint selective-prediction and online-control problem over calibrated early exits, remote split points, and a local full-model fallback.
- We introduce CODA-SC, which combines conformal and conformal-risk-control exit rules with a deadline- and energy-aware primal-dual controller.
- We establish finite-sample coverage and selective-error bounds for the exit rules and analyze an idealized binary-slack controller with $O(\sqrt{T})$ regret and deadline-violation bounds under

bounded feedback and latency-estimation error.

- We evaluate the method in a model-measured, trace-simulated benchmark with measured CNN FLOPs and activation payloads across four vision workloads, four network profiles, five seeds, and twelve policies, and add a learned bottleneck study that exposes the accuracy-latency trade-off of compressed split actions without claiming hardware deployment evidence.

2 Related Work

Split and collaborative inference. DDNNs [20] introduced a hierarchy in which end, edge, and cloud components can all participate in inference. Neurosurgeon [9] built layer-level latency and energy models to choose mobile-cloud partitions. Edgent [12] jointly considered partitioning and right-sizing through early exits under changing bandwidth. Auto-Split [4] turned collaborative edge-cloud AI into an automated deployment pipeline. Recent systems extend the design space to platform-agnostic partitioning [7], memory-limited devices [13], near-edge accelerators [14], and dynamic just-in-time edge execution [16]. These systems establish that partition choice must depend on compute and network state, but their decisions are typically made from profiled latency, bandwidth, or queueing models rather than calibrated prediction sets. When intermediate activations are larger than the input, learned bottlenecks such as BottleNet++ [18] compress the transmitted representation. CODA-SC is compatible with such compression: the main benchmark isolates control and calibration, and the additional bottleneck study tests whether compressed intermediate actions become active under the same controller.

Early exit and adaptive computation. Early-exit models attach intermediate classifiers to reduce average inference cost for easy samples [19]. Surveys identify the accuracy-latency trade-off and the need to jointly tune exits and split points [15]. Combining DNN partitioning with early exit adds flexibility but is often driven by performance models or confidence heuristics [6]. Calibration work argues that confidence alone is not the whole early-exit problem, because the decision should account for whether continuing computation is worth its cost [11]. CODA-SC differs by making the exit decision a conformal prediction-set decision. The advantage is not a claim of lower error than every confidence threshold; rather, it is an explicit finite-sample reliability statement and a controlled abstention mechanism that can be composed with online offloading.

Network robustness and online offloading. Packet-loss-tolerant split inference trains models to survive missing activation packets without retransmission [8]. AODPart [17] models partitioning with early exits and unknown communication/processing delay as online delay-constrained accuracy maximization. In-network split inference explores programmable network nodes and Named Data Networking under lossy edge connectivity [1]. CODA-SC targets a different failure mode: it couples a statistically calibrated exit rule with a deadline-aware controller and local fallback. This is complementary to compression, packet-loss robustness, and in-network placement. We compare directly against a reimplementation of the AODPart online-partitioning idea to separate the effect of online delay control from the effect of conformal selective prediction. This comparison is important because AODPart-style control can match CODA-SC on latency and deadline misses in some regimes; CODA-SC’s distinct contribution is the calibrated exit interface, the CRC selective-error variant, and explicit upload minimization through γb_k , not an unconditional improvement in every systems metric.

Foundation-model split inference. Large foundation models create new split-computing regimes, including dynamic edge orchestration [10] and WAN LLM split inference with speculative decoding [5]. These works show that wide-area networking is a first-order bottleneck. CODA-SC targets feed-forward classification in this manuscript; extending the method to autoregressive decoding requires new actions for token-level scheduling, KV-cache placement, and speculative execution.

Conformal prediction and risk control. Conformal prediction converts black-box model scores into prediction sets with finite-sample, distribution-free coverage under exchangeability [2]. Conformal risk control generalizes this idea to expected monotone losses [3]. CODA-SC uses conformal sets to decide when an intermediate prediction may safely leave the device as a point prediction, and uses conformal risk control to target selective error directly. The coverage statement is marginal and depends on exchangeability; the experiments therefore report calibration diagnostics rather than treating the theory as evidence of robustness under arbitrary distribution shift.

3 Problem Definition

We consider a request stream (X_t, Y_t) and a time-varying network process observed online. The exchangeability assumption used for conformal calibration applies to the labeled examples, not to the network process. A neural model has m local exits, K offloading split actions, and a local full-model action. Exit e produces class probabilities $p_e(y \mid x)$ after local prefix compute of a_e FLOPs. Split action k executes a local prefix of u_k FLOPs, uploads an activation of size b_k , and executes a cloud suffix of v_k FLOPs.

Latency and energy are derived from measured model FLOPs and activation payloads under a specified device model. Let the edge and cloud sustain Φ_e and Φ_c FLOP/s. For split k , the realized latency is

$$L_t(k) = \frac{u_k}{\Phi_e} + \frac{v_k}{\Phi_c} + R_t + Q_t + \frac{8b_k}{B_t} \cdot \frac{1}{1 - \ell_t}, \quad (1)$$

where B_t is uplink bandwidth in Mbps, R_t is round-trip time, Q_t is queueing delay, ℓ_t is loss probability, and b_k is measured in KB. A local exit at e has latency a_e/Φ_e and zero upload; the local full-model fallback has latency F/Φ_e where F is the full-model FLOPs. Edge energy of an action is $E(k) = u_k \varepsilon_e + b_k \varepsilon_{\text{up}}$, with ε_e the edge energy per FLOP and ε_{up} the upload energy per KB.

The online objective is to minimize a long-run cost combining latency, edge energy, and upload while satisfying a deadline budget. Let $M_t = \mathbb{1}\{L_t > D\}$ be the deadline miss. A target miss budget ρ requires

$$\frac{1}{T} \sum_{t=1}^T \mathbb{E}[M_t] \leq \rho. \quad (2)$$

For local exits, prediction reliability is controlled separately: the system emits a local point prediction only when the conformal set satisfies the exit rule. This separation is essential. A split action controls latency and energy but does not change the classifier’s uncertainty; an exit action controls communication and compute but must be statistically justified.

4 CODA-SC

4.1 Conformal Exit Calibration

For each exit e , define the nonconformity score

$$S_e(x, y) = 1 - p_e(y \mid x). \quad (3)$$

Given a calibration set $\mathcal{C} = \{(x_i, y_i)\}_{i=1}^n$ and target miscoverage α , CODA-SC computes

$$q_e = S_{e, (\lceil (n+1)(1-\alpha) \rceil)}, \quad (4)$$

the conformal quantile of calibration scores at exit e . At inference, exit e returns the prediction set

$$\Gamma_e(x) = \{y \in \mathcal{Y} : 1 - p_e(y \mid x) \leq q_e\}. \quad (5)$$

CODA-SC answers locally at the first e for which $|\Gamma_e(x)| = 1$; otherwise it invokes the online controller. This rule converts early exiting from an uncalibrated confidence comparison into a prediction-set decision. As an alternative that targets selective error directly, CODA-SC also supports a conformal-risk-control threshold q_e^* chosen so that the calibration singleton error, inflated by a finite-sample slack, is at most a target δ (Proposition 3).

4.2 Coupled Deadline-Aware Control

CODA-SC maintains exponential moving estimates $\hat{B}_t$, $\hat{R}_t$, and $\hat{Q}_t$ from recent offloaded requests. The estimated latency of split k is $\hat{L}_t(k) = u_k/\Phi_e + v_k/\Phi_c + \hat{R}_t + \hat{Q}_t + 8b_k/\hat{B}_t$, and the local fallback has fixed latency F/Φ_e . CODA-SC keeps a dual variable λ_t for deadline pressure and, over the action set $\mathcal{A}$ of split points *and* the local fallback, selects

$$k_t = \arg \min_{k \in \mathcal{A}} \hat{L}_t(k) + \lambda_t [\hat{L}_t(k) - D]_+ + \beta E(k) + \gamma b_k. \quad (6)$$

After observing realized latency $L_t(k_t)$, it updates

$$\lambda_{t+1} = [\lambda_t + \eta(\mathbb{I}\{L_t(k_t) > D\} - \rho)]_+. \quad (7)$$

The coupling to the exits has two functions. First, including the local fallback in $\mathcal{A}$ lets the same dual λ_t that tracks the deadline budget decide between offloading, which is energy-cheap but network-dependent, and local execution, which is deadline-safe but energy-expensive. Second, because local exits are admitted only through singleton conformal sets, the controller acts only on samples for which the intermediate classifiers did not provide a calibrated point prediction. The virtual queue increases the penalty on risky offloads after deadline misses and reduces it when recent requests satisfy the budget.

5 Theoretical Analysis

Assumption 1 (Exchangeable calibration). *For each exit e , the calibration examples and the future test example are exchangeable with respect to the nonconformity score $S_e(X, Y)$.*

Theorem 1 (Conformal exit coverage). *For any exit e calibrated as above, $\mathbb{P}\{Y \in \Gamma_e(X)\} \geq 1 - \alpha$.*

Algorithm 1 CODA-SC Inference

Require: exits $e = 1, \dots, m$, actions $\mathcal{A}$ (splits + local fallback), conformal quantiles $\{q_e\}$, deadline D , miss budget ρ

- 1: initialize network estimates $\hat{B}, \hat{R}, \hat{Q}$ and dual $\lambda \leftarrow 0$
- 2: **for** request x_t **do**
- 3: **for** exit $e = 1, \dots, m$ **do**
- 4: compute $p_e(\cdot \mid x_t)$ and $\Gamma_e(x_t) = \{y : 1 - p_e(y \mid x_t) \leq q_e\}$
- 5: **if** $|\Gamma_e(x_t)| = 1$ **then**
- 6: **return** $\arg \max_y p_e(y \mid x_t)$ locally
- 7: **end if**
- 8: **end for**
- 9: **for** action $k \in \mathcal{A}$ **do**
- 10: estimate $\hat{L}_t(k)$ and score $J_t(k) = \hat{L}_t(k) + \lambda[\hat{L}_t(k) - D]_+ + \beta E(k) + \gamma b_k$
- 11: **end for**
- 12: choose $k_t = \arg \min_k J_t(k)$; offload at split k_t or run the local fallback
- 13: observe latency $L_t(k_t)$; update $\hat{B}, \hat{R}, \hat{Q}$
- 14: $\lambda \leftarrow [\lambda + \eta(\mathbb{I}\{L_t(k_t) > D\} - \rho)]_+$
- 15: **end for**

Proof. This is the standard split-conformal classification result: under exchangeability the rank of the test score among the n calibration scores is uniform up to ties, so the $\lceil (n+1)(1-\alpha) \rceil$ calibration quantile yields marginal coverage at least $1-\alpha$. We state it to fix the exit interface; the contribution is its use for selective offloading, not the bound. $\square$

Proposition 1 (Single-exit false singleton bound). *Let $A_e = \{|\Gamma_e(X)| = 1\}$ and let $\hat{Y}_e$ be the singleton label. Then $\mathbb{P}(A_e \cap \{\hat{Y}_e \neq Y\}) \leq \alpha$, and if $\mathbb{P}(A_e) > 0$, $\mathbb{P}(\hat{Y}_e \neq Y \mid A_e) \leq \alpha/\mathbb{P}(A_e)$.*

Proof. When $\Gamma_e(X)$ is a singleton and the label is wrong, the true label is not in $\Gamma_e(X)$, so $A_e \cap \{\hat{Y}_e \neq Y\} \subseteq \{Y \notin \Gamma_e(X)\}$, whose probability is at most α by Theorem 1. Dividing by $\mathbb{P}(A_e)$ gives the conditional statement. $\square$

Proposition 2 (Sequential-exit false singleton bound). *If exit e is calibrated at level α_e and CODA-SC returns the first singleton among m exits, then the probability of returning any wrong singleton is at most $\sum_{e=1}^m \alpha_e$.*

Proof. A wrong accepted singleton at exit e implies $Y \notin \Gamma_e(X)$; the event of any wrong accepted singleton is a subset of $\cup_{e=1}^m \{Y \notin \Gamma_e(X)\}$, and a union bound with Theorem 1 gives the result. $\square$

Proposition 3 (Conformal-risk-control selective error). *Fix exit e , target δ , confidence level ζ , and a finite threshold grid $\mathcal{Q}$. Let $\hat{r}(q)$ be the empirical singleton selective error on $n_e(q)$ calibration singletons at threshold q . Choose the largest $q^* \in \mathcal{Q}$ satisfying*

$$\hat{r}(q^*) + \sqrt{\frac{\log(|\mathcal{Q}|/\zeta)}{2n_e(q^*)}} \leq \delta. \quad (8)$$

Then, with probability at least $1-\zeta$, the population singleton selective error at q^ is at most δ .*

Proof. For a fixed threshold, singleton correctness indicators are i.i.d. Bernoulli, so Hoeffding's inequality gives $\mathbb{P}(r(q) > \hat{r}(q) + \sqrt{\log(|\mathcal{Q}|/\zeta)/(2n_e(q))}) \leq \zeta/|\mathcal{Q}|$. A union bound over all $q \in \mathcal{Q}$ controls the data-dependent choice of q^* , and the defining inequality for q^* gives the claim. $\square$

Assumption 2 (Bounded feedback and feasibility). *The action set $\mathcal{A}$ is finite. For every action k and time t , the instantaneous cost $c_t(k) \in [0, C]$ and the constraint slack $g_t(k) = \mathbb{1}\{L_t(k) > D\} - \rho \in [-1, 1]$. There exists an admissible comparison policy π^* , using the same information as the controller, and slack $\epsilon > 0$ with $\mathbb{E}[g_t(\pi^*) \mid \mathcal{H}_{t-1}] \leq -\epsilon$ for every t (conditional Slater feasibility). Latency estimates satisfy $|\hat{L}_t(k) - L_t(k)| \leq \xi_t$; write $\bar{\xi} = \frac{1}{T} \sum_t \mathbb{E}[(1 + \lambda_t)\xi_t]$ for the weighted average latency-estimation error incurred by the controller.*

For the analysis, define the realized and estimated costs as $c_t(k) = L_t(k) + \beta E(k) + \gamma b_k$ and $\hat{c}_t(k) = \hat{L}_t(k) + \beta E(k) + \gamma b_k$, and define the binary deadline slacks as $g_t(k) = \mathbb{1}\{L_t(k) > D\} - \rho$ and $\hat{g}_t(k) = \mathbb{1}\{\hat{L}_t(k) > D\} - \rho$.

Scope of analysis. The theorem below analyzes the binary-slack primal-dual abstraction of CODA-SC, where deadline pressure enters through $\hat{g}_t(k)$. The implementation uses the hinge term $[\hat{L}_t(k) - D]_+$ in Eq. (6) as a graded surrogate so that actions far beyond the deadline are penalized more heavily than near-miss actions. The result therefore supports the constrained online-control mechanism under bounded feedback and latency-estimation error, while threshold flips for actions within ξ_t of D are absorbed into the $T\bar{\xi}$ term.

Theorem 2 (Idealized online deadline control). *Under Assumption 2, the controller of Eqs. (6)–(7), with the binary estimated slack $\hat{g}_t$ in the Lagrangian and step size $\eta = \Theta(T^{-1/2})$, satisfies, in expectation,*

$$\mathbb{E} \left[\sum_{t=1}^T (c_t(k_t) - c_t(\pi^*)) \right] \leq O(\sqrt{T}) + T\bar{\xi}, \quad \mathbb{E} \left[\sum_{t=1}^T (\mathbb{1}\{L_t(k_t) > D\} - \rho) \right] \leq O(\sqrt{T}) + T\bar{\xi}. \quad (9)$$

Proof. Let $\Phi(\lambda) = \frac{1}{2}\lambda^2$. Squaring Eq. (7) and using $[x]_+^2 \leq x^2$ and $|g_t| \leq 1$ gives the drift $\Phi(\lambda_{t+1}) - \Phi(\lambda_t) \leq \eta \lambda_t g_t(k_t) + \frac{1}{2}\eta^2$. Because k_t minimizes the estimated Lagrangian $\hat{c}_t(k) + \lambda_t \hat{g}_t(k)$ and the estimates deviate from the truth by at most ξ_t , for any fixed policy π we have $c_t(k_t) + \lambda_t g_t(k_t) \leq c_t(\pi) + \lambda_t g_t(\pi) + 2(1 + \lambda_t)\xi_t$. Adding η times this bound to the drift and telescoping over t with $\lambda_1 = 0$, $\Phi \geq 0$ yields

$$\sum_t (c_t(k_t) - c_t(\pi)) \leq \sum_t \lambda_t g_t(\pi) + \frac{1}{2}\eta T + 2 \sum_t (1 + \lambda_t)\xi_t.$$

Taking conditional expectations with $\pi = \pi^*$ and using conditional Slater feasibility gives $\mathbb{E}[\lambda_t g_t(\pi^*)] \leq -\epsilon \mathbb{E}[\lambda_t]$. This term is non-positive and, by the standard finite-action drift-plus-penalty stability lemma under Slater feasibility, the expected dual queue remains stable rather than requiring pointwise feasibility of π^* [21]. Therefore, with $\eta = \Theta(T^{-1/2})$, the expected cost regret is $O(\sqrt{T}) + 2 \sum_t \mathbb{E}[(1 + \lambda_t)\xi_t] = O(\sqrt{T}) + T\bar{\xi}$. For the constraint, $[x]_+ \geq x$ gives $\lambda_{t+1} \geq \lambda_t + \eta g_t(k_t)$, so summing and taking expectations yields $\mathbb{E}[\sum_t g_t(k_t)] \leq \mathbb{E}[\lambda_{T+1}]/\eta$. The same Slater stability argument bounds the expected terminal queue, giving $O(\sqrt{T})$ constraint violation, and the estimated-latency decisions add $T\bar{\xi}$. This is the standard constrained primal-dual/drift-plus-penalty result under Slater feasibility [21], specialized to a finite action set; the non-standard term is the additive estimation-error penalty from acting on $\hat{L}_t$ rather than L_t . $\square$

6 Experiments

6.1 Experimental Setup

We train a multi-exit convolutional network on four vision workloads that span a difficulty range: MNIST, Fashion-MNIST, CIFAR-10, and SVHN. The backbone is a compact VGG-style network

Table 1: Core experimental parameters. Deadlines are workload service-level objectives; device and energy values are modelling assumptions used to convert measured FLOPs and payloads into latency and energy.

Parameter	Value
Workload deadlines	MNIST 70 ms; Fashion-MNIST 75 ms; CIFAR-10 75 ms; SVHN 80 ms
Edge / cloud throughput	5 GFLOP/s edge; 5000 GFLOP/s cloud
Energy model	0.10 J/GFLOP edge compute; 5×10^{-4} J/KB upload
Calibration split	5000 held-out training examples per dataset and seed
Random seeds	7, 13, 29, 42, 101
Stream repeats and budget	6 repeats; deadline-miss budget $\rho = 0.08$

with three convolutional blocks (channel widths 64/128/256) and three early-exit heads after blocks 1–3, trained with a weighted multi-exit cross-entropy loss. Offloading actions are raw-input/cloud, split after block 1, split after block 2, split after block 3, and local full-model fallback. A held-out calibration split, drawn from the same data generation process as the test stream, is used for conformal and confidence thresholds.

The model-side quantities are measured from the trained networks. Activation payloads are exact int8 tensor sizes at each split point. Per-stage FLOPs are measured by forward hooks and converted to latency and energy through Eq. (1) using a documented device model: $\Phi_e = 5$ GFLOP/s for a constrained edge-CPU class and a server-GPU throughput for the suffix. This gives a local full-model latency of approximately 61 ms; early exits are cheaper, while offloading latency depends on the network trace. The throughput values are modelling assumptions, not hardware measurements, and are swept in Section 6.5. The network simulator produces Wi-Fi, LTE, congested, and mixed Markov traces, each with bandwidth, RTT, queueing, and loss. We evaluate five random seeds, repeat each test stream, use conformal miscoverage $\alpha = 0.10$, and set the deadline miss budget to $\rho = 0.08$.

For the compressed-split study, we freeze each trained CNN and train a lightweight codec at each intermediate split: a 1×1 convolution from the activation channels to 8 bottleneck channels, ReLU, and a 1×1 decoder back to the original activation dimension. The codec objective is the cloud-suffix cross-entropy plus 0.1 times activation-reconstruction mean-squared error. To emulate a high-resolution input regime while keeping the trained classifiers fixed, raw-input upload is scaled by 128 $\times$; the split payloads are not hand-scaled, but are the transmitted int8 bottleneck tensors. The latency model includes edge encoder FLOPs and cloud decoder FLOPs. This study tests whether the controller can use learned compressed representations; it is not a hardware measurement of codec execution.

6.2 Baselines and Metrics

We compare the two CODA-SC variants against local-only inference, cloud-only inference, static split, oracle split (per-request best split with trace knowledge but no early exit), confidence exit and BranchyNet-style confidence exit, deadline-greedy split, an AODPart-style online delay-constrained partitioning baseline, conformal exit without the coupled controller, and a conformal-risk-control exit. Metrics are accuracy, mean latency, P95 latency, deadline miss rate, upload volume, edge energy, local exit rate, and selective error among accepted local exits. We report means over four workloads, four profiles, and five seeds, with 95% confidence intervals and paired Wilcoxon signed-rank tests for the key comparisons.

Table 2: Aggregate results over four workloads, five seeds, and four Internet-like profiles. Lower is better for latency, misses, upload, energy, and selective error; higher is better for accuracy and exit rate.

Method	Acc. (%)	Mean ms	P95 ms	Miss (%)	Upload KB	Energy mJ	Exit (%)	Local (%)	Offload (%)	Sel. err. (%)
Local only	94.50	61.0	61.0	0.00	0.00	30.5	0.0	100.0	0.0	0.00
Cloud only	94.50	109.4	251.9	51.65	2.00	1.0	0.0	0.0	100.0	0.00
Static split	94.50	109.4	251.9	51.65	2.00	1.0	0.0	0.0	100.0	0.00
Oracle split	94.50	109.4	251.9	51.65	2.00	1.0	0.0	0.0	100.0	0.00
Confidence exit	93.04	28.9	51.6	1.67	0.09	12.7	96.8	0.0	3.2	5.64
BranchyNet exit	92.74	29.5	52.6	1.71	0.10	13.0	96.7	0.0	3.3	5.94
Deadline greedy	94.50	109.4	251.9	51.65	2.00	1.0	0.0	0.0	100.0	0.00
AODPart-style	93.04	27.2	42.1	0.06	0.02	13.4	96.8	2.4	0.8	5.64
Conformal exit	92.59	28.3	47.8	1.54	0.06	12.6	97.0	0.0	3.0	6.72
CRC exit	94.24	36.2	77.0	4.13	0.21	13.8	92.0	0.0	8.0	2.82
CODA-SC-Cov	92.59	26.8	48.5	0.05	0.01	13.2	97.0	2.2	0.7	6.72
CODA-SC-Risk	94.24	32.0	58.8	0.10	0.04	15.7	92.0	6.4	1.6	2.82

Policy implementation details. CODA-SC uses $\alpha = 0.10$, miss budget $\rho = 0.08$, energy weight $\beta = 6.0$, upload weight $\gamma = 0.05$, and dual step size $\eta = 0.03$ in the implemented hinge controller. Confidence-exit and BranchyNet-style baselines use calibration thresholds matched to a 5% selective-error target; the former thresholds maximum softmax probability and the latter thresholds entropy. The AODPart-style baseline is a mechanism-level reference for online delay-constrained partitioning, not an original-code reproduction: it does not reuse the published implementation or its implementation-specific tuning, but it preserves the relevant decision structure by using confidence exits when feasible, otherwise choosing the fastest estimated feasible partition, with local fallback when no partition is estimated to meet the deadline. The oracle split baseline uses realized trace latency to choose the best split action per request, but it does not use early exits or local fallback.

6.3 Results

Table 2 reports aggregate results. Selective local exits are the dominant latency mechanism: conformal exits answer most requests locally in tens of milliseconds, whereas cloud-only inference crosses the network on every request and averages 109 ms with 52% deadline misses. CODA-SC-Cov attains 26.8 ms mean latency and 0.05% deadline misses at 92.6% accuracy. The accuracy loss relative to full inference is approximately two percentage points and is the observed cost of frequent early exits on these four workloads. The device-model energy estimate decreases from 31 mJ for local-only inference to 13 mJ for CODA-SC. Table 3 shows that the reduction in deadline misses over conformal-exit is statistically significant ($p < 10^{-12}$, paired Wilcoxon). Relative to the AODPart-style baseline, CODA-SC-Cov has statistically comparable latency and deadline misses in this benchmark, lower upload, and an explicit conformal reliability interface. Thus, the empirical value of CODA-SC over AODPart-style control is not a blanket latency advantage; it is the combination of comparable deadline behavior, lower upload, calibrated exit coverage, and a separate risk-targeting variant. In this evaluation, the demonstrated systems effect is primarily calibrated local-exit selection plus deadline-aware handling of the uncertain residual stream. CODA-SC-Risk is the appropriate operating point when accuracy and selective error are prioritized: it raises accuracy from 92.6% to the CRC-exit level shown in Table 2, at the cost of higher mean latency and energy.

Calibrated exits answer a high fraction of requests across all four workloads, so the controller acts on a small residual of uncertain requests. That residual is disproportionately important for deadline reliability: when it is offloaded into a congested link, conformal-exit alone still misses 2.5%

Table 3: Headline metrics with 95% confidence intervals (over dataset×seed×profile cells) and paired Wilcoxon signed-rank p -values versus CODA-SC.

Method	Mean ms (95% CI)	Miss % (95% CI)	Upload KB (95% CI)	Sel. err. % (95% CI)	p vs Cov
Confidence exit	28.9±2.1	1.67±0.72	0.09±0.03	5.64±0.58	lat 4.0e-02, miss 3.3e-08
AODPart-style	27.2±1.5	0.06±0.04	0.02±0.02	5.64±0.58	lat 7.4e-02, miss 2.6e-01
Conformal exit	28.3±1.4	1.54±0.30	0.06±0.01	6.72±1.07	lat 3.3e-13, miss 3.3e-13
CODA-SC-Cov	26.8±1.3	0.05±0.02	0.01±0.01	6.72±1.07	–
CODA-SC-Risk	32.0±1.8	0.10±0.06	0.04±0.03	2.82±0.36	lat 1.1e-11, miss 1.6e-01

Table 4: Ablation. Conformal exits provide most of the latency and upload improvement; the coupled controller is responsible for driving deadline misses toward zero relative to conformal-exit alone.

Variant	Mean ms	P95 ms	Miss (%)	Upload KB	Energy mJ	Exit (%)	Sel. err. (%)
Deadline greedy	109.4	251.9	51.65	2.00	1.0	0.0	0.00
Confidence exit	28.9	51.6	1.67	0.09	12.7	96.8	5.64
Conformal exit	28.3	47.8	1.54	0.06	12.6	97.0	6.72
CODA-SC-Cov	26.8	48.5	0.05	0.01	13.2	97.0	6.72
CODA-SC-Risk	32.0	58.8	0.10	0.04	15.7	92.0	2.82

of deadlines on the congested profile. CODA-SC instead falls back to local computation for many of those requests and misses 0.0%, compared with 83% for cloud-only inference. This explains why the aggregate CODA-SC miss rate (0.05%) is far below conformal-exit (1.5%) despite similar exit rates and accuracy. The per-workload breakdown is provided in Supplementary Table S1; the same pattern holds across MNIST, Fashion-MNIST, CIFAR-10, and SVHN, with the largest residual latency and miss reductions appearing on the harder CIFAR-10 stream.

Supplementary Table S2 reports the controller’s residual action choices after local exits. It makes the split-computing limitation of the uncompressed benchmark explicit: the controller mainly chooses between raw offload and the local full-model fallback, while intermediate split actions are rarely selected because early activation tensors are not payload-efficient on these small-image workloads.

Table 5 replaces the earlier payload-only stress test with a learned-codec split study. Exits are disabled in this table so that the measured effect comes from split/offload/fallback actions rather than from local selective prediction. The split-only controller can now choose compressed intermediate actions whose predictions are produced by the decoded cloud suffix, not by the original full-model logits. The comparison therefore tests whether intermediate split actions become useful when the transmitted representation is learned rather than hand-scaled. In this regime, split-only policies move almost entirely to Split1 and reduce upload relative to raw cloud offload, but the learned codec also reduces accuracy. The full CODA controller still includes the local fallback, so its preference for local execution should be interpreted as a strict-deadline and accuracy-preserving control choice under the modelled edge throughput, not as a claim that learned splits are unnecessary in all regimes.

6.4 Confidence versus conformal exits

At matched operating points, confidence-threshold and conformal exits are competitive on accuracy and empirical selective error, and neither is uniformly superior across workloads. The distinguishing property of conformal exits is not lower empirical error but a finite-sample, distribution-free

Table 5: Learned-bottleneck split-selection study with exits disabled. Raw-input upload is scaled by $128\times$ to emulate larger frames; split actions use 8-channel learned 1×1 convolutional bottlenecks whose accuracy is computed from the decoded cloud suffix.

Method	Acc. (%)	Mean ms	P95 ms	Miss (%)	Upload KB	Local (%)	Raw (%)	Split1 (%)	Split2 (%)	Split3 (%)
Cloud only	94.50	403.8	939.5	91.05	256.00	0.0	100.0	0.0	0.0	0.0
Deadline greedy	80.53	125.1	267.6	65.11	2.00	0.0	0.0	100.0	0.0	0.0
Oracle split	80.55	125.1	267.6	65.10	2.92	0.0	0.7	99.2	0.1	0.0
CODA split controller (no exits)	80.53	125.1	267.6	65.11	2.00	0.0	0.0	100.0	0.0	0.0
CODA controller (no exits)	94.48	61.0	61.0	0.02	0.00	99.9	0.0	0.1	0.0	0.0

Table 6: Conformal calibration diagnostics averaged over datasets and seeds. Empirical coverage is close to the 90% target; the singleton rate quantifies how often a calibrated local decision is available.

Exit	Coverage (%)	Avg. set size	Singleton (%)	Singleton err. (%)
Layer 1	90.4	1.29	72.6	5.39
Layer 2	90.0	0.98	90.3	3.85
Layer 3	89.9	0.94	92.0	3.35

coverage result (Theorem 1) and an explicit selective-error control (Proposition 3) that a confidence threshold does not provide. The conformal-risk-control variant makes this explicit: it targets a selective-error level directly and trades exit rate for reliability accordingly. Thus the empirical claim is not that conformal exits are consistently faster or more accurate than tuned confidence exits, but that they expose a reliability contract that can be composed with deadline-aware control.

6.5 Sensitivity

Supplementary Table S3 sweeps the conformal level α and the edge throughput Φ_e . Smaller α lowers selective error but reduces the local exit rate, shifting more traffic to the controller; a weaker edge device raises local compute cost and therefore the value of offloading when the network is healthy. The qualitative conclusions are stable across the sweep, but the absolute deadline-miss rate is sensitive to edge throughput. This is expected: CODA-SC deliberately uses local execution as a deadline-safe fallback, so its benefit depends on the device being fast enough to complete the fallback before the deadline.

6.6 Discussion

The results establish a hierarchy of evidence. The main benchmark supports the calibrated selective-control mechanism: local conformal exits provide most of the latency and upload reduction, and the coupled controller suppresses deadline misses on the small residual stream by switching between offload and local fallback. It does not, by itself, demonstrate broad advantages for intermediate split selection. On these small-image workloads, early activation tensors can be larger than the raw input, so the controller often chooses between raw offloading and local computation rather than among many intermediate split points. This explains why cloud-only, static split, oracle-split, and deadline-greedy policies have similar aggregate latency in Table 2: without compression or larger inputs, intermediate split points do not create a meaningful payload advantage.

The learned-bottleneck study in Table 5 addresses a narrower question: whether the action space can express useful split decisions when intermediate representations are compressed by a

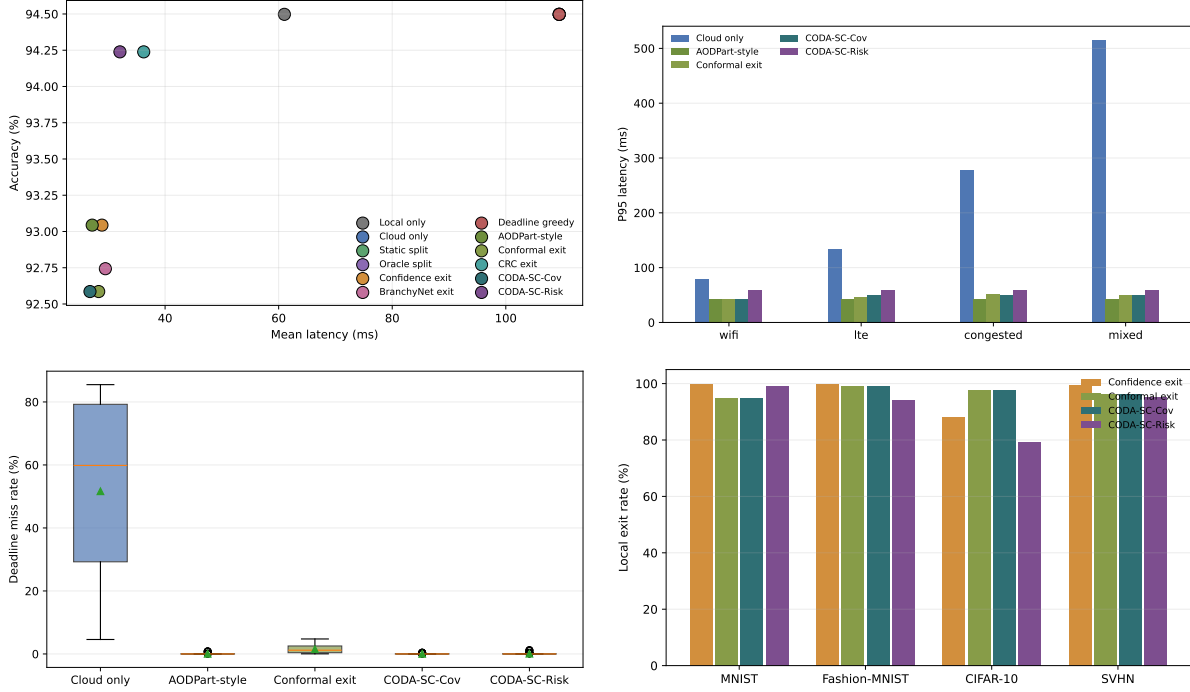


Figure 1: Top left: latency-accuracy trade-off averaged across profiles. Top right: profile-specific P95 latency for network-dependent methods. Bottom left: distribution of deadline-miss rates across dataset $\times$ seed $\times$ profile, showing that CODA-SC removes the congested-profile tail that cloud-only suffers. Bottom right: local exit rate by workload, which stays high (95–99%) across the difficulty range so that most requests avoid the network entirely.

trained codec. It strengthens the split-selection evidence relative to payload scaling alone because the accuracy of a split action is now produced by the decoded suffix. It also shows the cost of such compression: the split-only policies obtain lower upload and lower latency than raw cloud offload in the high-resolution regime, but at a substantial accuracy loss with this small codec. The present evidence should therefore be read in three levels: strong support for calibrated early exits and residual fallback/offload control; diagnostic support for learned compressed split actions and their accuracy-latency trade-off in the same simulated setting; and no claim of deployment-level hardware validation.

7 Limitations and Future Work

The benchmark is reproducible on standard CPU/GPU infrastructure, but it is not a hardware deployment study. Activation payloads and FLOPs are exact for the trained models; edge and cloud throughputs are documented modelling assumptions, and the sensitivity study shows that these assumptions materially affect deadline misses. Network latency is trace-simulated from bandwidth, RTT, queueing, and loss processes rather than measured on a live wide-area edge-cloud deployment, so operational effects such as diurnal load, routing changes, server contention, and correlated failures are not captured. The learned bottlenecks are lightweight 1×1 codecs trained on the same 32×32 vision workloads; they validate the compressed-action interface but are not a phone or edge-server measurement, and they do not replace a high-resolution deployment study with hardware

codec timing. The theoretical statements are marginal and depend on exchangeability of calibration and test examples; they do not by themselves protect against distribution shift, label shift, or adaptive adversarial inputs. Finally, the experiments are limited to feed-forward vision classification. Autoregressive foundation models introduce token-level scheduling, KV-cache placement, and speculative decoding decisions that require a different action space.

8 Conclusion

CODA-SC formulates split inference as a joint selective-prediction and online control problem. Conformal exits determine when an intermediate prediction is eligible for local output; a primal-dual controller determines how to handle the remaining requests under deadline, upload, and energy costs. On four multi-exit CNN vision workloads with simulated Internet-like traces, the method reduces mean latency and deadline misses relative to cloud-only and split-only policies, while exposing a reliability interface that confidence-only early exits lack. The gains are not free: frequent early exits reduce accuracy relative to full inference, the local fallback relies on sufficient edge throughput, and the calibration statements require exchangeability. The learned-bottleneck study shows how compressed intermediate actions can be evaluated with decoded-suffix predictions, but direct high-resolution and hardware experiments are still needed before claiming deployment-level split-selection gains. Future work should replace the device model with direct phone/edge measurements, evaluate adaptive conformal calibration under distribution shift, add stronger learned activation compression for high-resolution workloads, and extend the action space to autoregressive foundation-model inference.

Reproducibility Statement

All code needed for the experiments is included in this workspace. The main command is:

```
python3 scripts/run_experiments.py --seeds 7,13,29,42,101 --repeats 6 \
  --enable-learned-bottlenecks --bottleneck-channels 8 \
  --bottleneck-epochs 8 --output results
```

It trains the multi-exit CNNs (if not cached), trains the frozen-backbone split bottlenecks, measures the systems profiles, evaluates all policies over the network traces, and regenerates the CSV summaries, calibration diagnostics, LaTeX tables, figures, and graphical abstract included in this manuscript. Dataset splits, network traces, and initialization are controlled by fixed seeds.

Data Availability Statement

The code, cached model outputs, generated CSV summaries, LaTeX tables, figures, and diagnostics are available at <https://github.com/pz1004/coda-sc-public>. The experimental artifacts can be regenerated with the command in the reproducibility statement above. Public dataset sources are downloaded by the scripts where permitted by their original licenses; no proprietary data are used.

CRedit Author Statement

To be completed before submission with the final author names and roles (Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original

draft, Writing – review and editing, Visualization, Supervision, Project administration, and Funding acquisition as applicable).

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research received no external funding.

Declaration of Generative AI and AI-assisted Technologies in the Writing Process

During the preparation of this work the authors used OpenAI Codex for editorial revision planning, consistency checking, and manuscript editing support. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

- [1] Marica Amadeo, Claudia Campolo, Antonella Molinaro, and Giuseppe Ruggeri. In-network split inference with named data networking under lossy edge connectivity. In *2025 21st International Conference on Network and Service Management (CNSM)*, pages 1–6, 2025. doi: 10.23919/CNSM67658.2025.11297451.
- [2] Anastasios N. Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification, 2022.
- [3] Anastasios Nikolas Angelopoulos, Stephen Bates, Adam Fisch, Lihua Lei, and Tal Schuster. Conformal risk control. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=33XGfHLtZg>.
- [4] Amin Banitalebi-Dehkordi, Naveen Vedula, Jian Pei, Fei Xia, Lanjun Wang, and Yong Zhang. Auto-split: A general framework of collaborative edge-cloud ai. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 2543–2553, 2021. doi: 10.1145/3447548.3467078.
- [5] Michael Cunningham. Privacy-aware split inference with speculative decoding for large language models over wide-area networks, 2026.
- [6] Maryam Ebrahimi, Alexandre da Silva Veith, Moshe Gabel, and Eyal de Lara. Combining dnn partitioning and early exit. In *Proceedings of the 5th International Workshop on Edge Systems, Analytics and Networking*, pages 25–30, 2022. doi: 10.1145/3517206.3526270.
- [7] Soumendu Kumar Ghosh, Arnab Raha, Vijay Raghunathan, and Anand Raghunathan. Partner: Platform-agnostic adaptive edge-cloud dnn partitioning for minimizing end-to-end latency. *ACM Transactions on Embedded Computing Systems*, 23(1):1–38, 2024. doi: 10.1145/3630266.

- [8] Sohei Itahara, Takayuki Nishio, and Koji Yamamoto. Packet-loss-tolerant split inference for delay-sensitive deep learning in lossy wireless networks. In *2021 IEEE Global Communications Conference (GLOBECOM)*, pages 1–6, 2021. doi: 10.1109/GLOBECOM46510.2021.9685179.
- [9] Yiping Kang, Johann Hauswald, Cao Gao, Austin Rovinski, Trevor N. Mudge, Jason Mars, and Lingjia Tang. Neurosurgeon: Collaborative intelligence between the cloud and mobile edge. In *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 615–629, 2017. doi: 10.1145/3037697.3037698.
- [10] Fernando Koch, Aladin Djuhera, and Alecio Binotto. Intelligent orchestration of distributed large foundation model inference at the edge. *Computer Networks and Communications*, 2025. doi: 10.37256/cnc.3220256807.
- [11] Piotr Kubaty, Filip Szatkowski, Grzegorz Choczynski, Eric Nalisnick, and Bartosz Wojcik. Rethinking calibration for early-exit neural networks, 2026.
- [12] En Li, Liekang Zeng, Zhi Zhou, and Xu Chen. Edge ai: On-demand accelerating deep neural network inference via edge computing. *IEEE Transactions on Wireless Communications*, 19(1):447–457, 2020. doi: 10.1109/TWC.2019.2946140.
- [13] Xiangyu Li, Yuanchun Li, Yuanzhe Li, Ting Cao, and Yunxin Liu. Flexnn: Efficient and adaptive dnn inference on memory-constrained edge devices. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*, pages 709–723, 2024. doi: 10.1145/3636534.3649391.
- [14] Hao Liu, Mohammed E. Fouda, Ahmed M. Eltawil, and Suhaib A. Fahmy. Split dnn inference for exploiting near-edge accelerators. In *2024 IEEE International Conference on Edge Computing and Communications (EDGE)*, pages 84–91, 2024. doi: 10.1109/EDGE62653.2024.00020.
- [15] Yoshitomo Matsubara, Marco Levorato, and Francesco Restuccia. Split computing and early exiting for deep learning applications: Survey and research challenges. *ACM Computing Surveys*, 55(5):1–30, 2022. doi: 10.1145/3527155.
- [16] Motahare Mounesan, Xiaojie Zhang, and Saptarshi Debroy. Infer-edge: Dynamic dnn inference optimization in just-in-time edge-ai implementations. In *NOMS 2025 IEEE Network Operations and Management Symposium*, pages 1–9, 2025. doi: 10.1109/NOMS57970.2025.11073623.
- [17] Shiva Saxena and Ben Liang. Aodpart: Accuracy-optimal online partitioning for edge inference with delay constraint. In *Proceedings of the Twenty-sixth International Symposium on Theory, Algorithmic Foundations, and Protocol Design for Mobile Networks and Mobile Computing*, pages 211–220, 2025. doi: 10.1145/3704413.3764468.
- [18] Jiawei Shao and Jun Zhang. BottleNet++: An end-to-end approach for feature compression in device-edge co-inference systems. In *2020 IEEE International Conference on Communications Workshops (ICC Workshops)*, pages 1–6, 2020. doi: 10.1109/ICCWorkshops49005.2020.9145068.
- [19] Surat Teerapittayanon, Bradley McDanel, and H. T. Kung. Branchynet: Fast inference via early exiting from deep neural networks. In *2016 23rd International Conference on Pattern Recognition (ICPR)*, pages 2464–2469, 2016. doi: 10.1109/ICPR.2016.7900006.

- [20] Surat Teerapittayanon, Bradley McDanel, and H. T. Kung. Distributed deep neural networks over the cloud, the edge and end devices. In *2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS)*, pages 328–339, 2017. doi: 10.1109/ICDCS.2017.226.
- [21] Hao Yu and Michael J. Neely. A low complexity algorithm with $O(\sqrt{T})$ regret and $O(1)$ constraint violations for online convex optimization with long term constraints. *Journal of Machine Learning Research*, 21(1):1–24, 2020. URL <http://jmlr.org/papers/v21/16-494.html>.